RAG AND VECTOR DB

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

1. Introduction to RAG and Vector DB

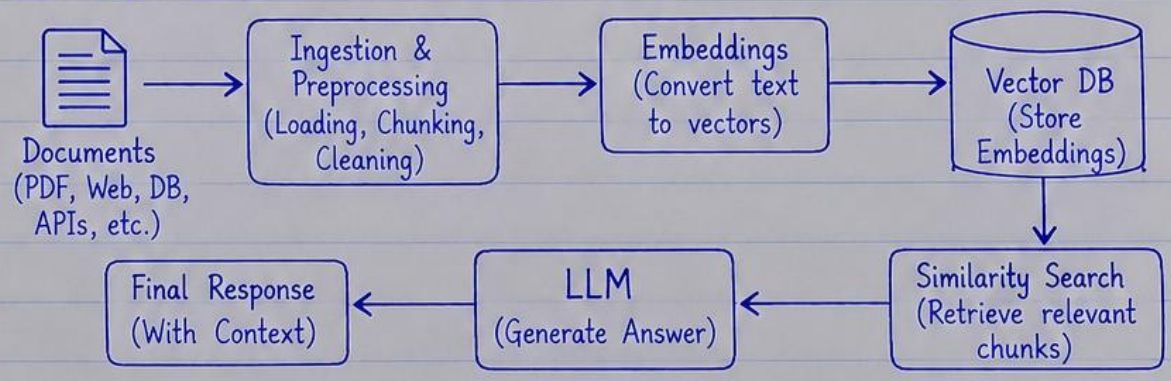
1.1 What is RAG ?

- RAG (Retrieval Augmented Generation) is a technique that combines information retrieval with LLMs to generate more accurate, relevant and up-to-date responses.
- It retrieves relevant information from external knowledge sources and provides it to the LLM as context.
- Helps in reducing hallucinations and allows LLMs to work with private or domain-specific data.

1.2 Why RAG is Important ?

- LLMs have knowledge cut-off and limited domain knowledge.
- RAG allows using latest and custom data (PDFs, websites, databases, etc.)
- Improves accuracy, relevance and credibility of responses.
- Useful for enterprise applications, internal documentation, chatbots, etc.

1.3 High Level Architecture of RAG



1.4 What is a Vector Database ?

- A Vector Database is a specialized database designed to store, manage and search vector embeddings efficiently.
- It allows similarity search to find semantically similar data.
- Popular Vector DBs: FAISS, Chroma, Pinecone, Weaviate, Qdrant, Milvus.

1.5 RAG vs Traditional LLM Approach

Aspect	Traditional LLM	RAG with Vector DB
Knowledge Source	Pretrained data (fixed)	External data (dynamic)
Answer Accuracy	May be outdated	More accurate and relevant
Domain Specific Data	Not possible	Easily possible
Hallucinations	Higher	Reduced
Use Cases	General Q&A	Enterprise, domain-specific, private data

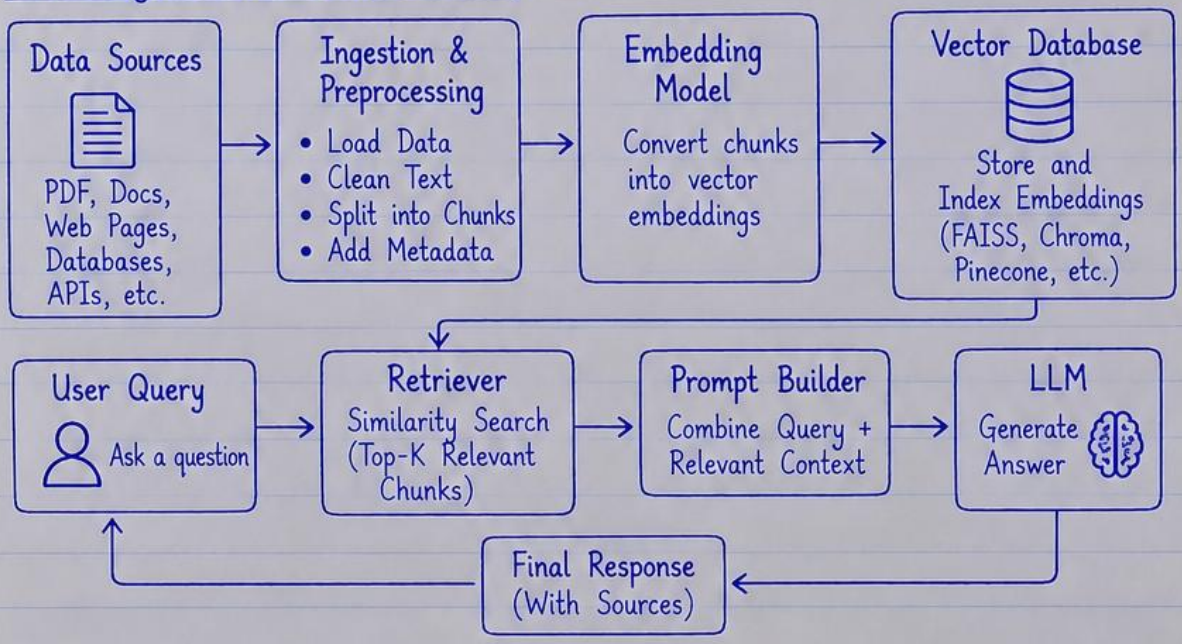
Rahul Yadav

2. RAG Architecture and Workflow

2.1 Overview

- RAG (Retrieval Augmented Generation) combines a retriever (Vector DB) with an LLM to generate more accurate and context-aware responses.
- It fetches relevant information from external documents and provides it to the LLM as context.
- This helps reduce hallucinations and allows using private or domain-specific data.

2.2 High Level Architecture



2.3 Step by Step Workflow

1. Load documents from various sources (PDF, Web, DB, etc.).
2. Preprocess and split documents into smaller chunks.
3. Convert each chunk into vector embeddings using an embedding model.
4. Store embeddings in a vector database with metadata.
5. When user asks a query, convert it into an embedding.
6. Search top-K similar chunks from the vector database.
7. Combine retrieved chunks with the query in a prompt.
8. Send the prompt to LLM to generate a final answer.
9. Return the response with references/sources (optional).

2.4 Key Components

Component	Purpose
Data Ingestion	Load and preprocess documents
Chunking	Split documents into smaller chunks
Embedding Model	Convert text chunks to vector embeddings
Vector Database	Store and search embeddings efficiently
Retriever	Find relevant chunks using similarity search
Prompt Builder	Combine query with retrieved context
LLM	Generate final response

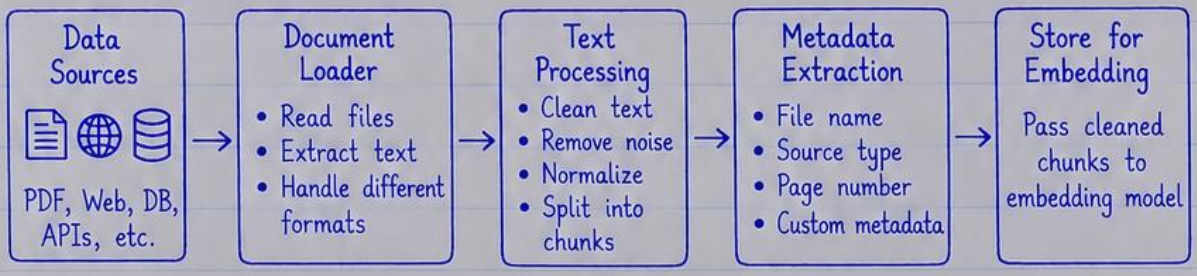
Rahul Yadav

3. Document Ingestion and Data Sources

3.1 Supported Data Sources

- PDF files (reports, manuals, research papers)
- Web pages (blogs, documentation, websites)
- Databases (MySQL, PostgreSQL, MongoDB, etc.)
- APIs (REST APIs, internal systems)
- Text files (TXT, MD, CSV, JSON, etc.)
- Cloud storage (S3, Google Drive, etc.)
- Enterprise tools (Confluence, Notion, SharePoint, etc.)

3.2 Ingestion Pipeline



3.3 Document Loaders in LangChain

Loader	Description
PyPDFLoader	Load text from PDF files
WebBaseLoader	Load content from a web page (URL)
DirectoryLoader	Load multiple files from a directory
CSVLoader	Load data from CSV files
UnstructuredFileLoader	Supports many file types (docx, pptx, etc.)
NotionLoader	Load data from Notion pages
ConfluenceLoader	Load data from Confluence

3.4 Example: Load PDF using LangChain

```
from langchain.document_loaders import PyPDFLoader

loader = PyPDFLoader("sample.pdf")
documents = loader.load()

print(f"Total Pages: {len(documents)}")
print(documents[0].page_content[:200])
```

→ Output:

```
Total Pages: 10
This is a sample PDF
content ....
```

3.5 Metadata Example

```
for doc in documents:
    print(doc.metadata)
```

→ Output:

```
{'source': 'sample.pdf',
 'page': 1,
 'file_path': 'data/sample.pdf'}
```

Rahul Yadav

4. Text Chunking and Preprocessing

4.1 Why Chunking is Important ?

- LLM models have a limited context window.
- Large documents need to be split into smaller chunks.
- Helps in better retrieval and more relevant answers.
- Improves search accuracy in vector databases.
- Avoids losing important context.

4.2 Chunking Strategies

Strategy	Description	When to Use
Fixed Size Chunking	Split text into fixed number of characters	Simple and fast
Recursive Chunking	Split based on headings, paragraphs, etc.	Best for documents with structure
Token Based Chunking	Split based on token count	When using LLM token limits
Semantic Chunking	Split based on meaning/embeddings	For better retrieval quality
Sliding Window Chunking	Create overlapping chunks	To maintain context between chunks

4.3 Chunking Process Flow



4.4 Examples of Chunking using LangChain

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=["\n\n", "\n", ".", " "]
)
chunks = text_splitter.split_text(text)
print(f"Total Chunks: {len(chunks)}")
print(chunks[0])
```

→ Output:

```
Total Chunks: 12
----------------
This is the first chunk
of the document ....
```

4.5 Preprocessing Techniques

- Remove extra spaces and new lines
- Clean special characters
- Remove headers, footers, page numbers
- Convert to lowercase (if applicable)
- Remove irrelevant content (ads, navigation, etc.)
- Maintain important structure (headings, tables)

Useful Libraries / Tools

- LangChain Text Splitters
- PyPDFLoader
- Unstructured (for complex files)
- tiktoken (for token counting)
- NLP libraries (spaCy, NLTK)

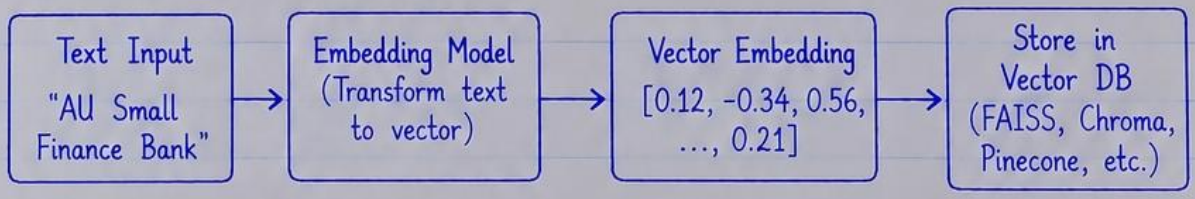
Rahul Yadav

5. Embeddings and Embedding Models

5.1 What are Embeddings ?

- Embeddings are numerical vector representations of text, documents, images, etc.
- They capture the semantic meaning of the content.
- Similar content will have similar vector representations.
- Embeddings help in similarity search, clustering and Retrieval Augmented Generation.

5.2 How Embeddings Work ?



5.3 Popular Embedding Models

Model Name	Provider	Dimensions	Best For	Notes
text-embedding-3-small	OpenAI	1536	General purpose	Fast, cost-effective
text-embedding-3-large	OpenAI	3072	High quality	Better accuracy
all-MiniLM-L6-v2	SentenceTransformers	384	Lightweight, fast	Good for small apps
bge-base-en	BAAI	768	High performance	Good for RAG
e5-base	Microsoft	768	Search & RAG	Strong retrieval
nomic-embed-text	Nomic	768	Long documents	Open source

5.4 Example: Generate Embeddings using LangChain (OpenAI)

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

text = "This is a sample document about banking."
vector = embeddings.embed_query(text)
print(len(vector))    # 1536
print(vector[:5])     # first 5 values
```

Output:

```
1536
[-0.0123, 0.0456, 0.0234, -0.0567, 0.0789]
```

5.5 Key Points

- Embeddings convert text into dense vectors.
- Use a good embedding model based on your use case.
- Higher dimension vectors can capture more information.
- Always use the same embedding model for indexing and querying.
- Embedding quality directly affects retrieval accuracy.
- You can also generate embeddings for files, chunks, and metadata.

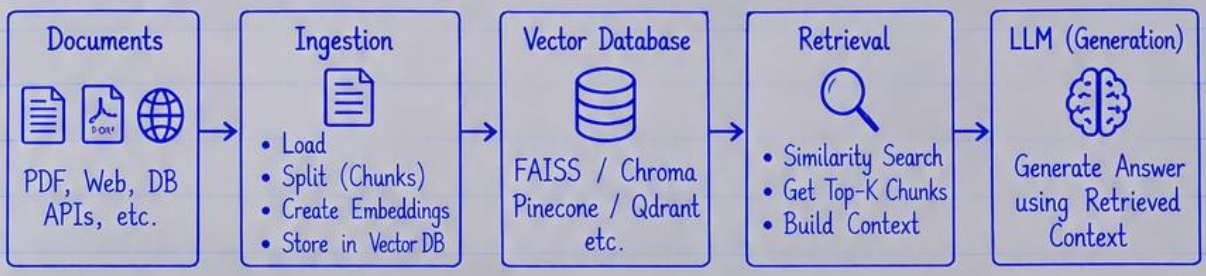
Rahul Yadav

8. Building a RAG Pipeline with LangChain

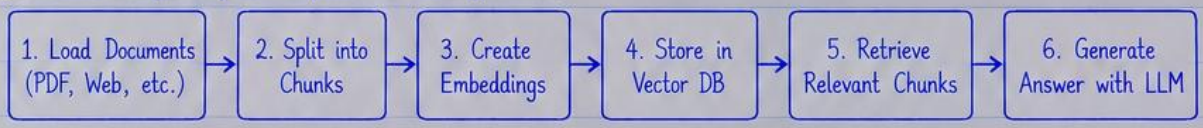
8.1 What is RAG Pipeline ?

- RAG (Retrieval Augmented Generation) combines retrieval and generation.
- It fetches relevant context from your data and passes it to an LLM.
- Helps in giving accurate and up-to-date answers using private data.
- Reduces hallucinations and improves response quality.

8.2 High Level Architecture



8.3 Step by Step RAG Pipeline



8.4 Example: End-to-End RAG using LangChain (FAISS)

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.llms import ChatOpenAI

# 1. Load and split document
loader = PyPDFLoader("au_bank_guide.pdf")
docs = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)

# 2. Create embeddings and store in FAISS
embeddings = OpenAIEmbeddings()
db = FAISS.from_documents(chunks, embeddings)

# 3. Create retrieval QA chain
llm = ChatOpenAI(model="gpt-3.5-turbo")
Qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=db.as_retriever(search_kwargs={"k": 3})
)

# 4. Ask a question
query = "What are the KYC documents required for savings account?"
result = qa_chain.run(query)
print(result)
```

Output (Sample Answer)

The KYC documents required for a savings account are:
1. Aadhaar Card
2. PAN Card
3. Address Proof (Utility Bill)
4. Recent Photograph

(Answer generated using your document data)

Key Points

- Uses your own documents as context.
- Supports PDF, web pages, databases, etc.
- Chunking + Embeddings + Vector DB + LLM.
- Improves accuracy and reduces hallucinations.
- Can be extended with metadata and filters.
- Works with different Vector DBs (FAISS, Chroma, Pinecone, Qdrant).

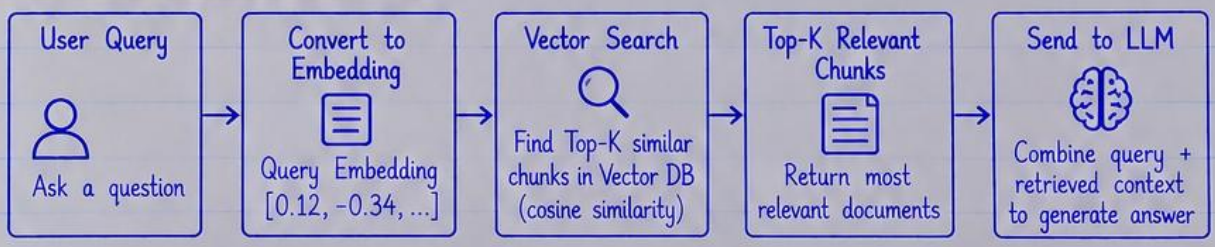
Rahul Yadav

7. Retrieval in RAG (Vector Search)

7.1 What is Retrieval ?

- Retrieval is the process of finding the most relevant document chunks from a vector database based on the user query.
- It uses the vector embedding of the query and compares it with stored embeddings.
- Top-K most similar chunks are returned as context for the LLM.
- Good retrieval significantly improves the quality and accuracy of the final response.

7.2 Retrieval Workflow



7.3 Similarity Search Types

Type	Description	When to Use	Notes
Similarity Search	Finds top-K most similar vectors using cosine/euclidean/dot	General RAG use case	Most common approach
MMR (Max Marginal Relevance)	Balances relevance and diversity	Avoid duplicate results	Improves answer quality
Hybrid Search	Combines vector search + keyword search (BM25)	Better accuracy for long documents	Used in production systems
Metadata Filtering	Filter results using metadata (e.g. source, date, file type)	Domain-specific retrieval	Improves relevance and security

7.4 Distance Metrics

Metric	Formula	Range	When to Use	Notes
Cosine Similarity	$\frac{A \cdot B}{\|A\| \times \|B\|}$	-1 to 1	Most common for text embeddings	Works well for normalized vectors
Euclidean Distance	$\sqrt{\sum (A_i - B_i)^2}$	0 to ∞	General purpose	Smaller value = more similar
Dot Product	$A \cdot B$	-∞ to ∞	If embeddings are normalized	Faster computation

7.5 Example: Similarity Search using LangChain (FAISS)

```
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings

db = FAISS.load_local("my_faiss_index", OpenAIEmbeddings())

query = "How to open a savings account?"
results = db.similarity_search(query, k=3)

for i, doc in enumerate(results):
    print(f"Result {i+1}: {doc.page_content[:100]}")
    print(f"Source: {doc.metadata['source']}")
    print("-----")
```

→ Output:

```
Result 1: AU Small Finance Bank allows you to open a savings account online using ...
Source: savings_account.pdf

Result 2: You can complete the account opening process ...
Source: account_opening_guide.pdf

Result 3: KYC documents required for savings account are ...
Source: kyc_docs.pdf
```

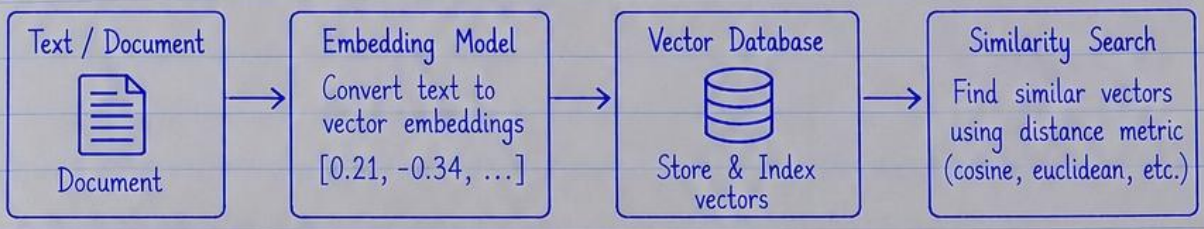
Rahul Yadav

6. Vector Databases in RAG

6.1 What is a Vector Database ?

- A Vector Database is a specialized database designed to store, manage and search vector embeddings efficiently.
- It enables similarity search to find semantically similar data.
- It is widely used in RAG, semantic search, recommendation systems, etc.
- Unlike traditional databases, it works with high-dimensional vectors.

6.2 How Vector Database Works ?



6.3 Popular Vector Databases

Vector DB	Type	Key Features	Best For	Notes
FAISS	Library (Meta)	Fast, efficient	Small to medium scale	In-memory, open source
Chroma	Open Source	Easy to use, persistent	RAG, prototypes	Good for developers
Pinecone	Managed (Cloud)	Scalable, high performance	Production apps	Paid service
Weaviate	Open Source	Hybrid search, GraphQL	Enterprise use cases	Supports modules
Qdrant	Open Source	High performance, filters	Production & large scale	Good for metadata filtering
Milvus	Open Source	Scalable, distributed	Large scale systems	Used in enterprise
Astra DB	Managed (Cloud)	Serverless, scalable	Production (Datastax)	Easy integration

6.4 Similarity Search and Distance Metrics

- Cosine Similarity : Most commonly used for text embeddings.
- Euclidean Distance : Measures straight line distance between vectors.
- Dot Product : Useful for normalized vectors.
- Higher similarity means more relevant content.

Cosine Similarity Formula

$$\text{similarity}(A,B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Range : -1 to 1
(1 = most similar)

6.5 Example : Using Chroma Vector DB (Python)

```python
import chromadb
from chromadb.utils import embedding_functions

# Create client
client = chromadb.PersistentClient(path="./chroma_db")

# Create collection
collection = client.get_or_create_collection(name="my_docs")

# Add documents
collection.add(
    documents=["AU Small Finance Bank is an Indian bank.",
               "It provides banking and financial services."],
    ids=["doc1", "doc2"]
)
```

```python
# Query similar documents
results = collection.query(
    query_texts=["banking services"],
    n_results=2
)
print(results["documents"])
```

Output :

```
[["It provides banking and
  financial services.',
  'AU Small Finance Bank
  is an Indian bank."]]
```

Rahul Yadav

9. Chunking Strategies in RAG

9.1 What is Chunking ?

- Chunking is the process of splitting large documents into smaller, meaningful pieces (chunks) before creating embeddings.
- It helps in better context retrieval, higher accuracy and lower token usage.
- Good chunking strategy significantly improves RAG performance.
- Chunk size and overlap play an important role in maintaining context.

9.2 Why Chunking is Important ?

Better Retrieval



Finds more relevant and accurate chunks

Handles Large Documents



Splits big files into manageable pieces

Maintains Context



Overlap keeps context between chunks

Optimizes Token Usage



Avoids sending unnecessary large text to LLM

9.3 Popular Chunking Strategies

Strategy	Description	Best For	Pros	Cons
Fixed Size	Split by fixed number of characters	Simple documents	Easy to implement	May break sentences
Recursive	Splits using structure (headers, paragraphs, sentences)	General purpose	Maintains context	Slightly complex
Semantic	Uses meaning (vector similarity)	Long, technical docs	Better context preservation	Requires embeddings
Token Based	Split by token count (using tiktoken)	LLM specific use case	Controls token size	May break structure
Paragraph Based	Split by paragraphs	Articles, blogs, docs	Natural chunks	Paragraph size varies

9.4 Chunking with LangChain (Python)

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text = """
(Your document text here...)
"""

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,        # number of characters
    chunk_overlap=200,      # overlap between chunks
    separators=["\n\n", "\n", ".", " "]
)
chunks = splitter.split_text(text)
print(f"Total chunks: {len(chunks)}")
print(chunks[0])
```

Parameters

- chunk_size : Size of each chunk (e.g. 500, 1000 characters)
- chunk_overlap : Overlap between consecutive chunks (e.g. 100, 200)
- separators : List of separators to break text (paragraph, sentence, etc.)
- length_function : Function to measure length (default: len)

Example Output

```
Total chunks: 12
Chunk 1 : "AU Small Finance Bank
          provides banking and financial
          services ..."
```

9.5 Tips for Better Chunking

- Use 500-1000 characters per chunk.
- Keep 10-20% overlap to maintain context.
- Use document structure (headings, paragraphs).
- Avoid very small or very large chunks.
- For technical documents, use recursive or semantic chunking.
- Experiment with different chunk sizes and evaluate retrieval quality.

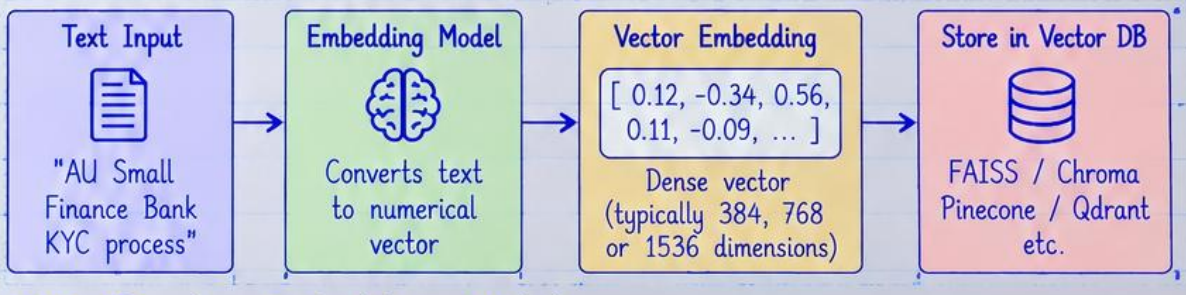
Rahul Yadav

10. Embeddings in RAG

10.1 What are Embeddings ?

- Embeddings are numerical vector representations of text, images or other data.
- They capture the semantic meaning of the content in a dense vector format.
- Texts with similar meaning have similar vectors (closer in vector space).
- Embeddings are used in RAG to store and retrieve relevant chunks from a Vector DB.

10.2 How Embeddings Work ?



10.3 Popular Embedding Models

Model	Provider	Dimensions	Best For	Notes
all-MiniLM-L6-v2	SentenceTransformers	384	General purpose	Lightweight, fast
text-embedding-3-small	OpenAI	1536	High quality	Good for most use cases
text-embedding-3-large	OpenAI	3072	Better accuracy	More powerful
bge-base-en	BAAI	768	General purpose	Open source
instructor-xl	HuggingFace	768	Instruction based	Good for RAG tasks

10.4 Generating Embeddings (Python Example)

Using SentenceTransformers (all-MiniLM-L6-v2)

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')
text = "AU Small Finance Bank KYC process"
embedding = model.encode(text)
print("Embedding dimension:", len(embedding))
print(embedding[:10])        # first 10 values
```

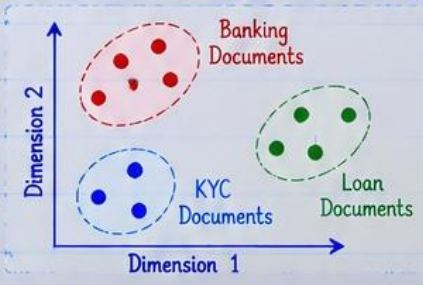
Sample Output

```
Embedding dimension: 384
[  0.0123, -0.0345, 0.0567, 0.0112,
  -0.0987, 0.0432, 0.0065, -0.0211,
   0.0334, -0.0176, ... ]
```

Using OpenAI Embeddings

```
from openai import OpenAI
client = OpenAI(api_key="your-api-key")
response = client.embeddings.create(
    model="text-embedding-3-small",
    input="AU Small Finance Bank KYC process"
)
embedding = response.data[0].embedding
print(len(embedding))   # 1536
```

10.5 Visualizing Embeddings in Vector Space



- Similar documents are placed closer in vector space.
- Different topics form different clusters.
- This helps in retrieving relevant documents using similarity search.

10.6 Key Takeaways

- Embeddings convert text into numerical vectors.
- Use a good embedding model based on your use case.
- Store embeddings in a Vector DB for similarity search.
- Similar meaning texts have similar vectors.
- Embedding quality directly impacts RAG performance.

Rahul Yadav

11. Vector Databases in RAG

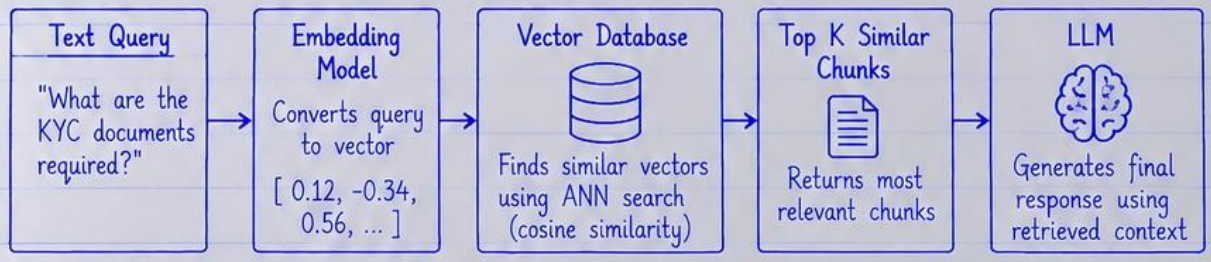
11.1 What is a Vector Database ?

- A Vector Database is a specialized database designed to store, manage and search high dimensional vectors (embeddings).
- It enables fast similarity search to find semantically similar data.
- Unlike traditional databases, it focuses on vector operations and similarity metrics (cosine similarity, dot product, euclidean distance).

11.2 Key Features

- Efficient storage and indexing of high dimensional vectors.
- Fast similarity search (Approximate Nearest Neighbor - ANN).
- Supports metadata filtering along with vector search.
- Scales to millions or billions of vectors.
- Widely used in RAG, recommendation systems, semantic search, image search, etc.

11.3 How Vector Search Works ?



11.4 Popular Vector Databases

Database	Description	Best For	Key Features	Notes
FAISS	Open-source by Meta	Research, large scale	Fast, efficient, ANN	In-memory, high performance
Chroma	Open-source	Small to medium projects	Easy to use, persistent storage	Popular for RAG prototypes
Pinecone	Managed cloud service	Production systems	Scalable, high availability	Paid service
Qdrant	Open-source	Production systems	Fast, filtering, payload support	Good for metadata filtering
Weaviate	Open-source	Semantic search applications	GraphQL, hybrid search	Supports multiple modalities
Milvus	Open-source	Large scale applications	Scalable, high performance	Used in enterprise

11.5 Similarity Metrics

Metric	Formula / Idea	Use Case	Notes
Cosine Similarity	Measures angle between vectors	Most common for text embeddings	Value range: -1 to 1
Dot Product	Direct vector multiplication	When vectors are normalized	Faster than cosine
Euclidean Distance	Straight line distance between vectors	Useful for some applications	Lower distance = more similar

11.6 Key Takeaways

- Vector databases store and search embeddings efficiently.
- They enable semantic search and are essential for RAG.
- Choose a database based on your scale, use case and feature requirements.
- Cosine similarity is most commonly used for text embeddings.
- Metadata filtering and hybrid search improve retrieval quality.

Rahul Yadav

12. Chroma DB in RAG

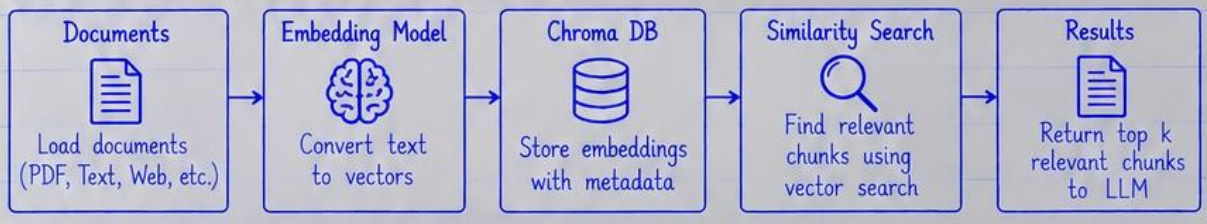
12.1 What is Chroma DB ?

- Chroma DB is an open-source vector database designed for AI applications.
- It stores vector embeddings and provides fast similarity search.
- It is lightweight, easy to use and can run locally or in the cloud.
- Widely used in RAG systems, prototypes and production applications.

12.2 Key Features

- Open source and easy to set up.
- Supports metadata filtering.
- Persistent storage (local or cloud).
- Fast similarity search.
- Integrates well with LangChain and other frameworks.
- Good for development, research and small to medium scale applications.

12.3 How Chroma DB Works ?



12.4 Basic Example (Using LangChain + Chroma)

Create and Store Embeddings

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

loader = TextLoader("data.txt")
documents = loader.load()
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
           chunk_overlap=100)
docs = splitter.split_documents(documents)

embeddings = OpenAIEmbeddings()
db = Chroma.from_documents(
         documents=docs,
         embedding=embeddings,
)        persist_directory="./chroma_db"
```

Query the Vector Store

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()
db = Chroma(persist_directory="./chroma_db",
            embedding_function=embeddings)

query = "What are the KYC documents required?"
results = db.similarity_search(query, k=3)

for i, doc in enumerate(results):
    print(f"Result {i+1}:", doc.page_content)
    print("Metadata:", doc.metadata)
    print("---")
```

12.5 Chroma DB vs Other Vector Databases

Feature	Chroma	FAISS	Pinecone	Qdrant
Type	Open Source	Open Source	Managed Cloud	Open Source
Setup	Easy (Local/Cloud)	Manual Setup	Cloud Only	Easy (Local/Cloud)
Metadata Filtering	Yes	Limited	Yes	Yes
Scalability	Small to Medium	Medium	High	High
Best For	RAG, Prototypes	Research	Production	Production

12.6 Key Takeaways

- Chroma DB is a simple and powerful vector database for RAG.
- It is easy to integrate with LangChain.
- Supports metadata filtering and persistent storage.
- Good choice for development and small to medium scale applications.
- For large scale production, consider managed solutions like Pinecone or Qdrant.

Rahul Yadav

14. Evaluation Metrics for RAG Systems

14.1 Why Evaluation is Important ?

- Helps to measure the quality, accuracy and usefulness of RAG responses.
- Ensures the retrieved documents are relevant and the generated answer is correct.
- Useful for comparing different models, vector databases and retrieval techniques.
- Helps in continuous improvement and monitoring in production.

14.2 RAG Evaluation Areas

Retrieval Evaluation
- Are the right documents retrieved ?
- Is the ranking good ?

Generation Evaluation
- Is the answer accurate ?
- Is the answer factually correct ?

End-to-End Evaluation
- Does the final response satisfy the user query ?
- Is it helpful and relevant ?

System Evaluation
- Latency
- Cost
- Scalability
- User Satisfaction

14.3 Common Evaluation Metrics

Metric	Type	What it Measures ?	Range	Notes
Precision	Retrieval	% of retrieved docs that are relevant	0 - 1	Higher is better
Recall	Retrieval	% of relevant docs that are retrieved	0 - 1	Higher is better
Hit Rate @ K	Retrieval	Whether at least one relevant doc is in top K results	0 - 1	Higher is better
MRR (Mean Reciprocal Rank)	Retrieval	Average of reciprocal rank of first relevant doc	0 - 1	Higher is better
Context Precision	Generation	Relevance of context used by LLM	0 - 1	Higher is better
Faithfulness	Generation	Answer is grounded in retrieved context	0 - 1	Higher is better
Answer Relevance	Generation	How relevant the answer is to the query	0 - 1	Higher is better
BLEU / ROUGE	Generation	Similarity with reference answer	0 - 1	Higher is better
Latency	System	Time taken to generate response	ms / sec	Lower is better
Cost per Query	System	Total cost of retrieval + generation	$	Lower is better
User Satisfaction	System	Human feedback / rating	1 - 5	Higher is better

14.4 Example: Retrieval Evaluation (Hit Rate @ K)

User Query

"What is the KYC process?"

→

Top K Retrieved Documents

1. KYC Guide.pdf (Relevant)
2. Account Opening.pdf (Relevant)
3. FD Policy.pdf (Not Relevant)
4. Loan Policy.pdf (Not Relevant)
5. KYC FAQ.pdf (Relevant)

→

Hit Rate @ 5

Relevant documents found in top 5 = 3

Hit Rate @ 5 = 3/5 = 0.6

14.5 Example: Generation Evaluation (Faithfulness)

Retrieved Context

"KYC is completed online using Aadhaar, PAN and live video verification..."

→

Generated Answer

"KYC can be completed online using Aadhaar, PAN and video verification."

→

Faithfulness Check
- Based on retrieved context
- No hallucination
- Faithfulness = 1.0 (High)

14.6 Key Takeaways

- Use a combination of retrieval, generation and system level metrics.
- For RAG, retrieval metrics (Recall, Precision, Hit Rate) are very important.
- Use faithfulness and relevance metrics to ensure accurate and grounded answers.
- Monitor latency and cost for production systems.
- Use human feedback to evaluate real world performance.

Rahul Yadav

15. Optimizing RAG (Latency, Cost, Caching, Prompt Optimization)

15.1 Why Optimization is Important ?

- RAG systems can be slow and costly if not optimized.
- Optimization helps in improving response time, reducing cost and increasing accuracy.
- It ensures a better user experience and makes the system scalable for production.
- Proper optimization can significantly improve the overall performance of RAG pipeline.

15.2 Optimization Areas Overview

Latency Optimization	Cost Optimization	Caching	Prompt Optimization	Other Techniques
Faster response time	Reduce LLM and embedding cost	Store and reuse frequent results	Better prompts for accurate answers	Improved chunking, reranking, model selection, compression, etc.

15.3 Latency Optimization Techniques

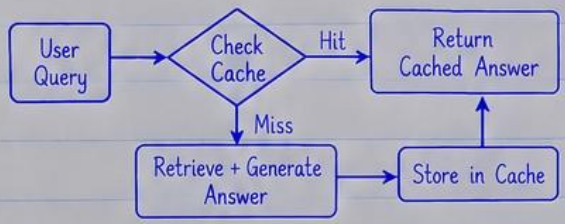
Technique	Description	Benefit
Efficient Chunking	Use optimal chunk size and overlap	Reduces processing time
Vector Index Optimization	Use appropriate index type (HNSW, IVF, PQ)	Faster similarity search
Asynchronous Processing	Perform embedding and indexing in background	Non-blocking pipeline
Streaming Response	Stream LLM output to user	Better user experience
Use Smaller / Faster Models	Use lightweight embedding and LLM models	Lower latency

15.4 Cost Optimization Techniques

Technique	Description	Benefit
Use Efficient Embedding Models	Use models like all-MiniLM-L6-v2	Lower embedding cost
Cache Embeddings	Avoid re-embedding same documents	Save computation cost
Use Smaller LLM Models	Use GPT-3.5 / open-source models for simple queries	Reduce LLM cost
Limit Retrieved Context	Send only top relevant chunks	Lower token usage
Use Compression / Summarization	Summarize long documents before sending to LLM	Reduce input tokens

15.5 Caching in RAG

- Cache frequent queries and their answers.
- Cache embeddings of documents to avoid re-computation.
- Use Vector DB built-in caching or external cache (Redis).
- Helps in reducing latency and overall cost.



15.6 Prompt Optimization Tips

- Use clear and specific instructions.
- Provide context and constraints.
- Use few-shot examples if required.
- Keep the prompt concise and relevant.
- Avoid irrelevant or unnecessary information.
- Iterate and test different prompt templates.

Example Prompt Template

"You are a helpful assistant. Use the following context to answer the question.
If the answer is not available in the context, then say 'I don't know'.
Context: {context}
Question: {user_query}
Answer:"

15.7 Key Takeaways

- Optimizing RAG improves performance, reduces cost and increases accuracy.
- Use efficient chunking, indexing and caching strategies.
- Choose the right models and limit the amount of context sent to the LLM.
- Prompt optimization plays a crucial role in getting better and relevant answers.
- Monitor and continuously improve based on real world usage and feedback.

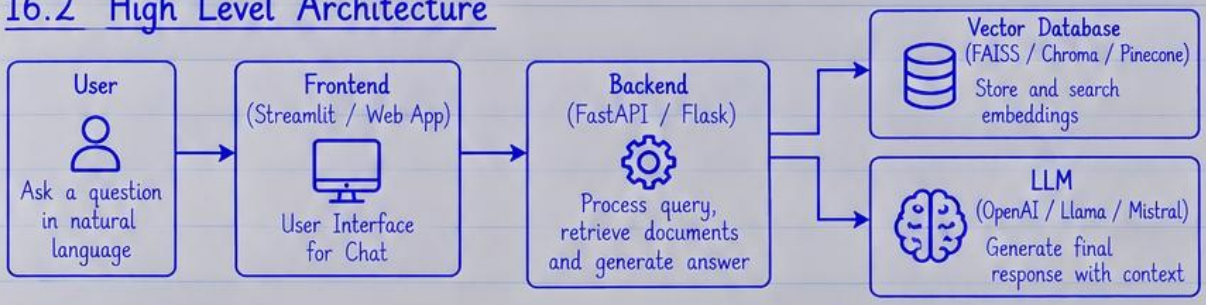
Rahul Yadav

16. Real-World Projects
(Chat with PDF, Knowledge Bot, Internal Docs, etc.)

16.1 Project Overview

- Build a RAG based application that allows users to chat with their private documents. (PDF, Word, internal docs, websites, etc.).
- The system retrieves relevant information from documents and generates accurate answers using LLM.
- Useful for enterprise knowledge base, internal documentation, customer support, etc.
- Can be extended with authentication, chat history and source citations.

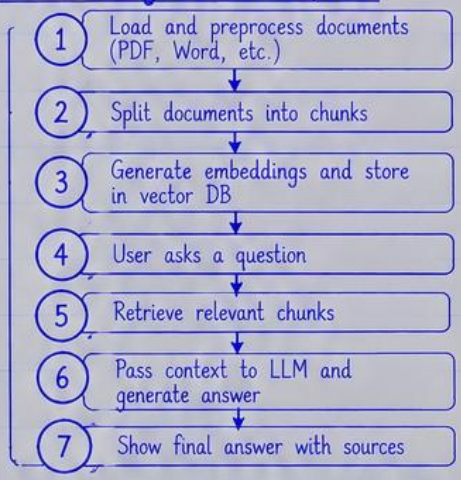
16.2 High Level Architecture



16.3 Tech Stack

Component	Technology Options
Frontend	Streamlit, React, Next.js
Backend	FastAPI, Flask
Document Processing	PyPDF2, Unstructured, LangChain Document Loaders
Text Splitter	RecursiveCharacterTextSplitter
Embeddings	OpenAI Embeddings, SentenceTransformers
Vector Database	FAISS, Chroma, Pinecone, Qdrant
LLM	OpenAI GPT, Llama, Mistral, etc.
Deployment	Docker, Kubernetes, Cloud (AWS / GCP)

16.4 Project Workflow



16.5 Example: Ingest PDF and Store in Chroma

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

# Load PDF
loader = PyPDFLoader("data/report.pdf")
documents = loader.load()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
docs = splitter.split_documents(documents)

# Create embeddings and store in Chroma
embeddings = OpenAIEmbeddings()
db = Chroma.from_documents(
        documents=docs,
        embedding=embeddings,
        persist_directory="./chroma_db"
)
print("Documents stored successfully!")
```

16.6 Example: Query the Knowledge Base

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

# Load existing vector store
embeddings = OpenAIEmbeddings()
db = Chroma(persist_directory="./chroma_db",
            embedding_function=embeddings)

# Create retriever and QA chain
retriever = db.as_retriever(search_kwargs={"k": 3})
llm = OpenAI(temperature=0.2)
qa_chain = RetrievalQA.from_chain_type(
            llm=llm,
            retriever=retriever,
            return_source_documents=True
)
query = "What is the KYC process?"
result = qa_chain({"query": query})
print("Answer:", result["result"])
print("Sources:", result["source_documents"])
```

16.7 Key Takeaways

- RAG can be used to build powerful real-world applications like document chat, knowledge bots, etc.
- Choose the right document loader, chunking strategy and vector DB based on your use case.
- Streamlit/FastAPI makes it easy to build and deploy POC quickly.
- Always show source documents to improve trust and transparency.
- Can be extended with authentication, analytics, citations, and feedback.

Rahul Yadav

17. Deployment and Scalability (Cloud, Docker, Kubernetes)

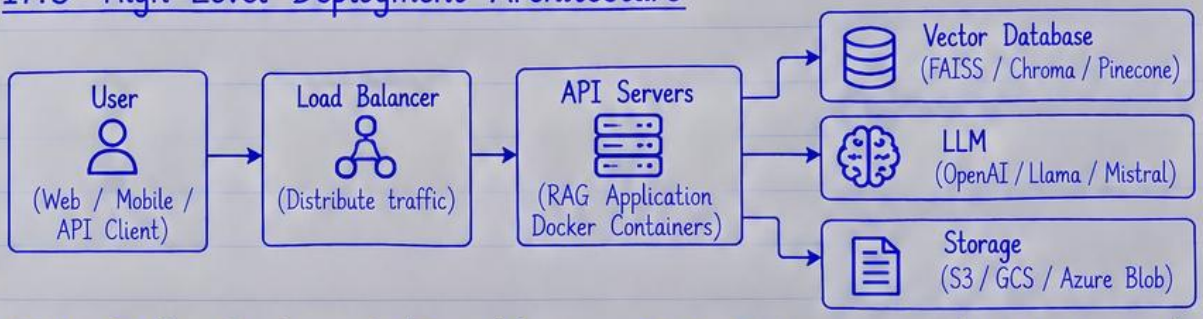
17.1 Why Deployment is Important ?

- Makes the RAG application available to real users.
- Ensures scalability, reliability and high availability.
- Helps in handling large documents and multiple concurrent users.
- Supports monitoring, logging and easy updates.
- Can be deployed on cloud or on-premise based on security requirements.

17.2 Deployment Options

Option	Description	Best For	Pros	Cons
Local (Dev)	Run on local machine	Development / Testing	Easy and quick	Not scalable
Docker	Containerize application	Small to medium apps	Portable, consistent	Single node
Docker Compose	Run multiple containers	Development / Staging	Easy setup	Not for large scale
Kubernetes	Container orchestration	Production / Large scale	Scalable, self-healing	Complex setup
Cloud (AWS/GCP/Azure)	Managed infrastructure	Production	Highly scalable, reliable	Cost can be high

17.3 High Level Deployment Architecture



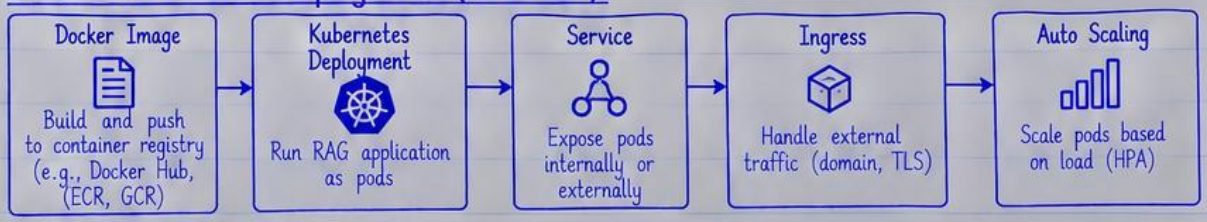
17.4 Docker Deployment (Example)

```
# Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.port=8501",
     "--server.address=0.0.0.0"]
```

17.5 Docker Compose (Multiple Services)

```
# docker-compose.yml
version: '3.8'
services:
  app:
    build: .
    ports:
      - "8501:8501"
    depends_on:
      - chroma
  chroma:
    image: chromadb/chroma
    ports:
      - "8000:8000"
    volumes:
      - ./chroma_data:/chroma/chroma
```

17.6 Kubernetes Deployment (Overview)



17.7 Key Takeaways

- Use Docker to containerize your RAG application.
- Use Docker Compose for local development with multiple services.
- Use Kubernetes for production to achieve scalability and high availability.
- Choose cloud providers like AWS, GCP or Azure for managed and scalable infrastructure.
- Implement monitoring, logging and auto-scaling for production systems.

Rahul Yadav

18. Best Practices, Common Issues and Troubleshooting

18.1 Best Practices for RAG Systems

- Use high quality and relevant documents.
- Apply proper chunking and metadata.
- Choose the right embedding model and vector database.
- Optimize retrieval and prompt for better accuracy.
- Implement caching, monitoring and logging.
- Keep the system secure and scalable.
- Continuously evaluate and improve based on user feedback.

18.2 Best Practices Across the RAG Pipeline

1. Data Ingestion
- Use reliable sources
- Clean and preprocess
- Extract metadata
- Remove irrelevant content

2. Chunking
- Use semantic or recursive chunking
- Keep optimal chunk size
- Preserve context
- Add metadata

3. Embeddings
- Use latest embedding models
- Normalize embeddings
- Handle updates and re-indexing

4. Retrieval
- Use hybrid search (keyword + vector)
- Tune top_k value
- Apply reranking
- Filter using metadata

5. Generation
- Use clear prompts
- Provide context
- Set temperature
- Add source citations
- Handle edge cases

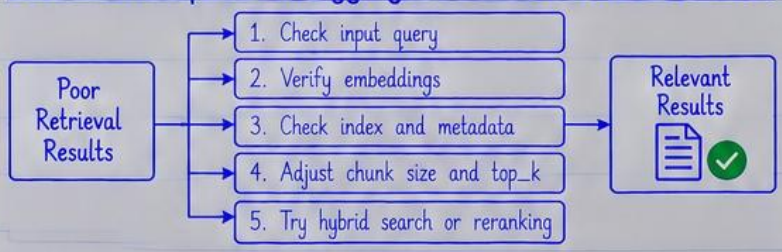
18.3 Common Issues and Solutions

Issue	Possible Cause	Solution
Irrelevant results	Wrong chunking or low quality data	Improve chunking and use better data sources
Low accuracy	Missing context or poor prompt	Add more relevant context and optimize prompt
Slow response	Large documents or no caching	Use caching, smaller chunks and efficient DB
High cost	Sending too much context to LLM	Use top_k, summarization and smaller models
Hallucinations	Insufficient or irrelevant context	Improve retrieval and add source citations
Vector DB issues	Wrong index type or incorrect query	Check index settings and query parameters
Deployment failures	Docker / Kubernetes misconfiguration	Check logs, resource limits and environment variables

18.4 Troubleshooting Guide

Problem	Steps to Troubleshoot	Example Commands / Tips
No results from vector DB	Check embeddings, index and query	Verify embedding dimension and index type
Poor retrieval quality	Check chunking, embeddings, metadata	Try different chunk size and top_k values
High latency	Check network, DB and model response time	Use caching and monitor API latency
Docker container not running	Check logs and dependencies	docker logs <container_id>
Kubernetes pod crash	Check pod logs and resource limits	kubectl logs <pod-name>
Model not generating answer	Check prompt and context length	Reduce context and try simpler prompt

18.5 Example: Debugging Poor Retrieval Results



Debugging Checklist
- Is the query clear and specific?
- Are embeddings generated correctly?
- Is the right index being used?
- Is metadata added and filtered?
- Have you tried different top_k values?
- Do you need hybrid search or reranking?

18.6 Key Takeaways

- Follow best practices at every stage of the RAG pipeline.
- Identify and troubleshoot common issues quickly.
- Use monitoring, logging and evaluation to improve the system.
- Optimize for accuracy, latency and cost.
- Keep iterating based on real world usage and feedback.

Rahul Yadav

19. Interview Questions and Key Takeaways

19.1 Common Interview Questions (with Short Answers)

Question	Short Answer
1. What is RAG ?	RAG combines information retrieval with LLMs to generate more accurate and relevant responses using external data.
2. Why do we need Vector DB ?	To store and search embeddings efficiently for semantic search.
3. How does similarity search work ?	Convert text to vector embeddings and find nearest vectors using distance metrics (cosine, L2, etc.).
4. What are common embedding models ?	OpenAI (text-embedding-3-small), all-MiniLM-L6-v2, BGE, E5, etc.
5. Difference between FAISS and Chroma ?	FAISS is a low-level vector search library, Chroma is an easy-to-use vector database with metadata filtering.
6. How to handle large documents ?	Use chunking (fixed/recursive/semantic) with appropriate chunk size and overlap.
7. What is hybrid search ?	Combination of keyword search (BM25) and vector search to improve retrieval accuracy.
8. How to reduce hallucinations ?	Use relevant context, proper chunking, reranking, source citations and temperature control.
9. How to evaluate RAG system ?	Use metrics like Precision, Recall, Hit Rate @ K, MRR, Faithfulness, Answer Relevance, BLEU/ROUGE.
10. How to deploy a RAG application ?	Use Docker containers, Kubernetes for scaling, cloud providers (AWS/GCP/Azure) and monitoring with logs/metrics.

19.2 Scenario Based Interview Questions

Scenario	What to Do ?
1. Low accuracy in answers	Check chunking, embeddings, vector DB settings, try hybrid search.
2. Slow response time	Use caching, smaller models, optimize DB queries, use async processing.
3. Need domain specific knowledge	Ingest domain documents, use metadata filtering, fine-tune prompts.
4. Large PDF files	Use recursive/semantic chunking, store embeddings with metadata.
5. Real-time data required	Combine RAG with live APIs or database queries.
6. Want to show sources	Store document metadata and return source citations with answer.

19.3 Coding / Hands-on Interview Questions

1. Create embeddings and store in Chroma

```
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma

embeddings = OpenAIEmbeddings()
db = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings
})
db.add_texts([
    "KYC process document....",
    "Account opening guide...."
])
print("Documents added to Chroma!")
```

2. Perform similarity search and get results

```
query = "What is the KYC process?"
results = db.similarity_search(query, k=3)

for i, doc in enumerate(results):
    print(f"Result {i+1}:")
    print("Content:", doc.page_content)
    print("Metadata:", doc.metadata)
    print("\n")
```

19.4 Key Takeaways

- RAG is a powerful approach to build knowledge-based applications.
- Understand retrieval, embeddings, vector databases and LLM integration.
- Be ready to discuss design decisions, trade-offs and real world challenges.
- Hands-on coding knowledge with LangChain, Vector DBs and deployment is important.
- Focus on accuracy, scalability, cost optimization and user experience.

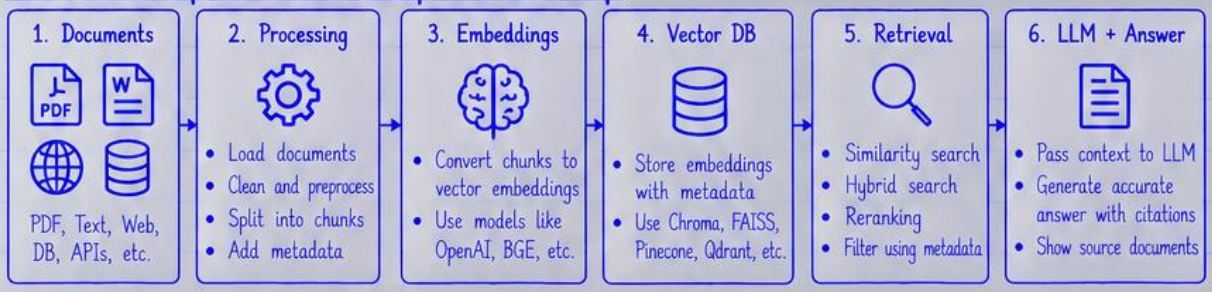
Rahul Yadav

20. Summary and Next Steps

20.1 What We Learned ?

- Understood what RAG is and why it is important.
- Learned about vector databases and their use cases.
- Explored the complete RAG pipeline from data ingestion to answer generation.
- Implemented practical examples using LangChain and Chroma.
- Understood deployment options using Docker and Kubernetes.
- Covered best practices, common issues and troubleshooting.
- Solved real world projects and interview questions.

20.2 Complete RAG Pipeline Recap



20.3 Key Tools and Technologies

Component	Tools / Options	Purpose
Document Loaders	LangChain Document Loaders	Load data from PDF, Web, DB, APIs, etc.
Text Processing	RecursiveCharacterTextSplitter	Split, clean and create chunks
Embedding Models	OpenAI, BGE, all-MiniLM-L6-v2	Convert text to vectors
Vector Databases	Chroma, FAISS, Pinecone, Qdrant, Weaviate	Store and search embeddings
RAG Frameworks	LangChain, LlamaIndex	Build and manage RAG pipeline
LLMs	OpenAI GPT, Llama, Mistral, etc.	Generate final response
Deployment	Docker, Kubernetes, AWS, GCP, Azure	Scale and run in production
Frontend (Optional)	Streamlit, FastAPI, Flask	Chat interface for users

20.4 Best Practices Checklist

- ✅ Use high quality and relevant documents
- ✅ Clean and preprocess data properly
- ✅ Choose the right chunking strategy
- ✅ Add useful metadata
- ✅ Select the right embedding model
- ✅ Tune retrieval parameters (top_k, filters, reranking)
- ✅ Use caching to reduce latency
- ✅ Monitor performance and user feedback
- ✅ Keep security and access control in mind
- ✅ Evaluate the system regularly

20.5 Common Pitfalls to Avoid

- ❗ Using irrelevant or low quality documents
- ❗ Incorrect chunking (too large or too small)
- ❗ Not adding metadata
- ❗ Using wrong embedding model
- ❗ Ignoring retrieval parameters
- ❗ Not handling large documents properly
- ❗ High latency due to no caching
- ❗ Sending too much context to LLM
- ❗ Not checking index type and DB settings
- ❗ Skipping monitoring and evaluation

20.6 Next Steps / What to Learn Next ?

1. Build more real world projects (company docs, customer support bot, etc.)
2. Explore advanced techniques (Graph RAG, Agentic RAG, Multi-Modal RAG)
3. Try other vector databases (Pinecone, Qdrant, Weaviate)
4. Learn LlamaIndex and compare with LangChain
5. Explore fine-tuning and custom embedding models
6. Deploy on cloud with CI/CD pipeline
7. Add authentication and access control
8. Implement monitoring and analytics
9. Work on domain specific use cases (finance, healthcare, HR, etc.)
10. Keep practicing interview questions and coding examples

Rahul Yadav